ARCHITECTURE & GUIDE BOOK

FreelanceOS

Complete Working, Request Flow, and Architecture Reference Manual

This document explains the step-by-step working of FreelanceOS SaaS platform — from frontend interactions to database commits, detailing every request-response cycle and backend background processes.

Prepared for: Developer onboarding & Reference

Stack: MERN (MongoDB, Express, React, Node.js) + Socket.io + Cron + Puppeteer

Created: July 2026

1. System Architecture Overview (Project Ka Structure)

FreelanceOS ek **full-stack SaaS application** hai jo freelancers ko unki poori workflow manage karne me help karta hai. Yeh application ek **Monorepo Architecture** follow karta hai jisme clear separator hai client-side (Frontend) aur server-side (Backend) ke beech me:

📁 Workspace Directory Structure

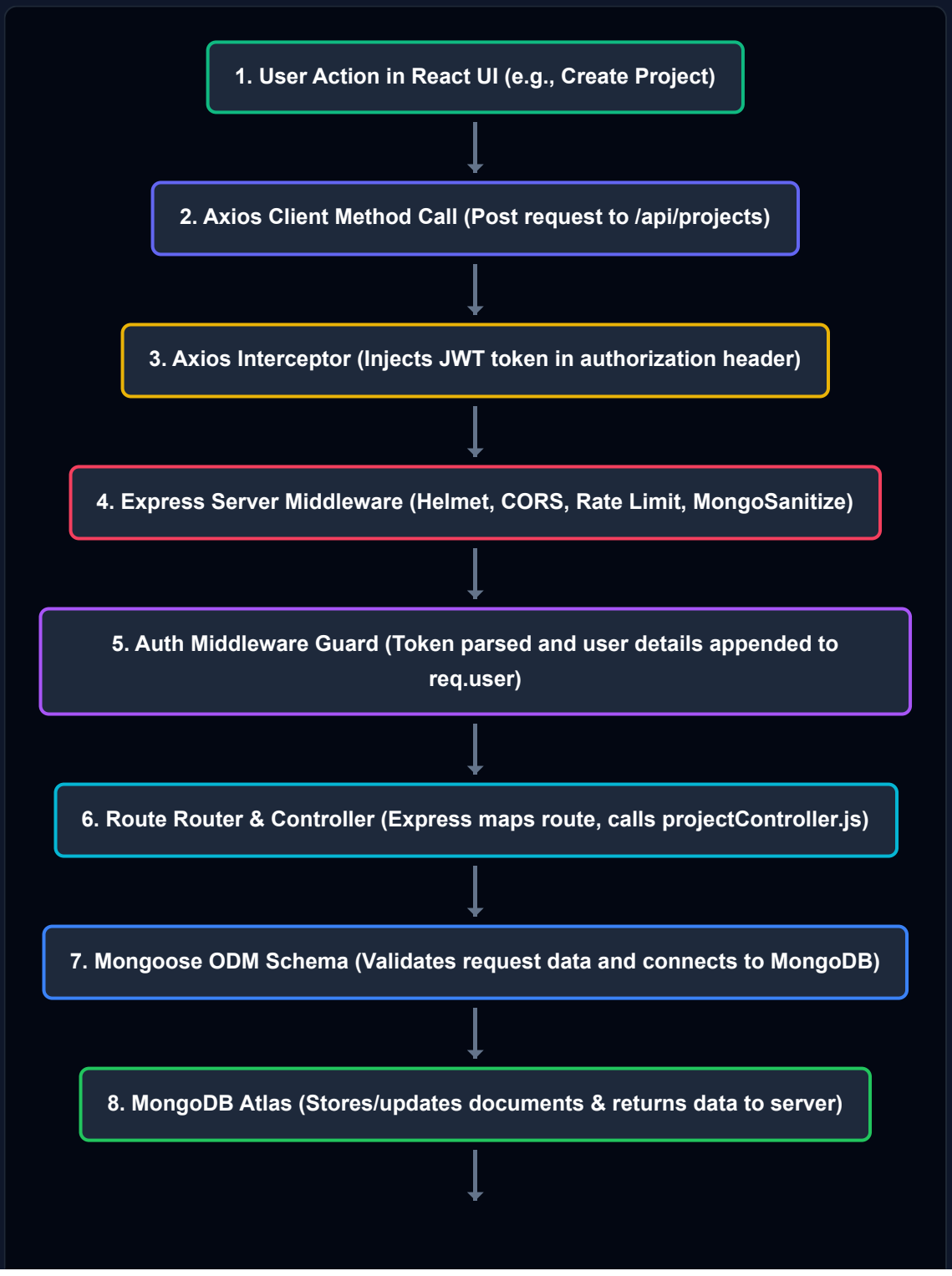
```
freelancer-saas/
├── backend/                # Express.js API Server + Socket.io + Cron jobs
│   ├── controllers/        # Business Logic / Controllers (DB calls & data validat
│   ├── models/             # Mongoose Schemas (Database collection definitions)
│   ├── routes/             # API Endpoints Route Mapping
│   ├── middleware/         # Auth guards, security checks & file-upload configurat
│   ├── utils/              # Helper utilities (Nodemailer wrappers, custom helpers
│   └── server.js           # Server Entry Point (Express + Socket.io Server + Cron
│
└── frontend/               # React client app built with Vite
    ├── src/
    │   ├── pages/          # 20+ pages (Dashboard, Projects, Kanban, Invoices, Cli
    │   ├── components/     # Reusable UI parts (Layout Shell, Sidebar, Button, For
    │   ├── store/          # Zustand global state (Auth status, user details, toke
    │   └── lib/            # Axios instance configuration & HTTP Interceptors
    └── vite.config.js      # Vite build & bundler configuration
```

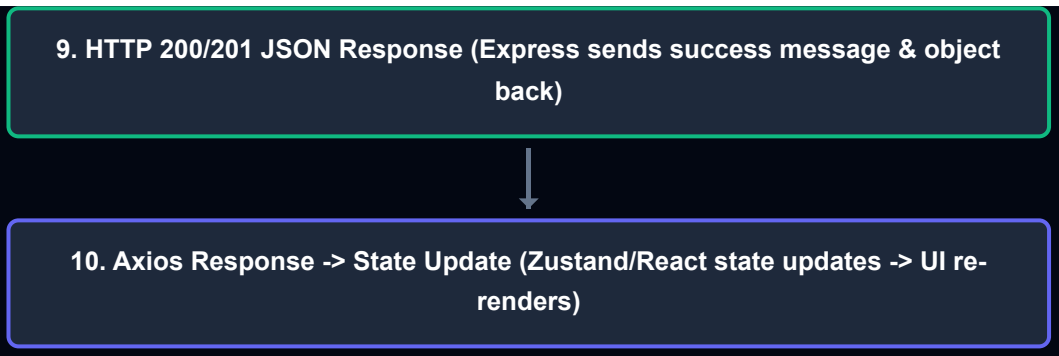
Frontend Tech Stack: React 19 (UI Rendering), Vite (Dev Server & Bundling), Zustand (Global State Management for authentication), Axios (HTTP Client with interceptor), Tailwind CSS (Styling), Framer Motion (Page transitions).

Backend Tech Stack: Node.js & Express (REST API), MongoDB & Mongoose (ODM & Database), Socket.io (Real-time Websockets), Node-cron (Background Cron jobs), Puppeteer (PDF generator engine), Razorpay (Payment gateway).

2. End-to-End Request & Response Flow (Kaise Request Flow Karti Hai)

React Browser App se lekar MongoDB Database tak, jab user koi action leta hai toh request kaise travel karti hai, use niche diagram aur text me detail me explain kiya gaya hai:




9. HTTP 200/201 JSON Response (Express sends success message & object back)
10. Axios Response -> State Update (Zustand/React state updates -> UI re-renders)

🔍 Step-by-Step Detail (Beginner Explanation)

1. **User Action:** User screen par ek form fill karke click karta hai (Jaise "Create Project" or "Login").

2. **Axios Request:** Frontend me humne `frontend/src/lib/axios.js` me ek custom Axios instance banaya hai jo API Requests handle karta hai. Form submit hone par yeh code API path par request bhejta hai:

```
// Example call from Projects page:
const response = await axiosInstance.post('/projects', formData);
```

3. **Axios Request Interceptor (Injecting Credentials):** Agar route protected hai, toh browser me active user ka JWT Token authorization headers me automatically inject ho jata hai via interceptor:

```
// Appended automatically:
config.headers.Authorization = `Bearer ${token}`;
```

4. **Express Middleware Chain:** Backend me `backend/server.js` par request pahunchte hi wo security and validator filters se pass hoti hai:
   - **Helmet:** Secures HTTP headers to protect against common attacks.
   - **CORS:** Verifies the request origin (checks if it comes from the authorized frontend URL).
   - **express-rate-limit:** Limiting API flooding.
   - **express-mongo-sanitize:** Prevents NoSQL Injection by sanitizing parameters starting with "$" or ".".

5. **Authentication Middleware Guard (verify JWT):** Agar API route lock hai (jaise projects, invoices, clients), toh request `backend/middleware/auth.js` me `protect()` middleware check se pass hoti hai:
   - Token ko header se read kiya jata hai, secret key se decode aur verify kiya jata hai.
   - Mongoose se user collection search ki jati hai, aur query user database row fetch karke `req.user` parameter me store kar deti hai.
   - Agar route admin-only hai (e.g. creating invoice), toh `adminOnly()` middleware check karta hai ki `req.user.role === 'admin'` hai ya nahi.

6. **Route Router & Controller Mapping:** Server request ko match routing rule ke sath target controller par bhejta hai:

```
// In backend/routes/projects.js
router.post('/', protect, adminOnly, createProject);
```

Yeh call flow direct transition karta hai `backend/controllers/projectController.js` ke `createProject` function par.

7. **Database Interaction (Mongoose & MongoDB):** Controller me validation execute hoti hai, data sanitize hota hai aur model interface ke through MongoDB connection par operations execute kiye jate hain:

```
const project = new Project({ ...req.body, freelancer: req.user.id });
await project.save();
```

8. **JSON Response:** Mongoose save success hone par controller response bhejta hai:

```
res.status(201).json({ success: true, data: project });
```

9. **Frontend UI Update:** Frontend ka Axios client response resolve karta hai, React page component local state ya Zustand state update karta hai, aur Screen automatically custom micro-animations ke sath update ho jati hai.

3. Key Modules & Deep-Dive Architecture (Important Features Kaise Kaam Karte Hain)

FreelanceOS me kuch advanced engineering blocks hain, unka execution flow niche simplify karke samjhaya gaya hai:

🔑 Authentication & OTP Verification Flow

FreelanceOS user authentication ke liye password hashing aur safe authentication tokens (JWT) ka use karta hai:

- **Register:** `/api/auth/register` par email/password se account create hota hai. Password database me save hone se pehle `bcryptjs` (12 salt rounds) ke standard encryption algorithm se hash ho jata hai.
- **OTP Generation & Nodemailer:** Registration ke sath, 6-digit random OTP verify code generate hota hai aur database me expiration date (typically 10-15 mins) ke sath save ho jata hai. Ek transactional email Nodemailer SMTP wrapper ke through user ke registered inbox me send ho jata hai.
- **Verify OTP:** User code enter karta hai. Backend database OTP match hone par field status `isVerified = true` kar deta hai. Jab tak account verify nahi hota, `PrivateRoute` guards redirect blocks trigger karte rehte hain.
- **Login:** User credentials check hone par user model database search validation pass karta hai. Server custom encryption keys (JWT_SECRET) ke through ek token code generates karta hai jo client application access token ke roop me local storage me state ke andar capture ho jata hai.

💬 Real-Time Collaboration & Socket.io Room Architecture

Project page chats, invitations alerts aur instant notification ke liye bidirectional WebSockets network `Socket.io` integrated hai:

```
// 1. Connection establishment:
// Client registers its presence to the Socket.io WebSocket server
socket.emit('join-room', currentUserId);

// 2. targeted Room communication:
```

```
// Backend controllers use room scope to emit alerts dynamically
const io = req.app.get('io');
io.to(targetUserRoomId).emit('new-message', messagePayload);
```

Room Isolation Model: Jab connection build hota hai, tab backend connection listener server client-side ko authorize karke user specific ID ke base par private socket session channel `room` create kar deta hai. Jab bhi developer background dynamic activity updates broadcast karta hai, query data direct single active targeted socket connection pipeline me flow karta hai.

⏰ Background Automation Engine (Cron Jobs)

Cron jobs automatic routine tasks background processes schedule karte hain. `backend/server.js` file initialization stage me direct scheduling config setup load karti hai:

- **Recurring Invoice Generator (Midnight Cron: `0 0 * * *`):** Daily midnight check run hota hai. System check karta hai ki konse invoices me `isRecurring: true` hai aur unka `nextInvoiceDate ≤ current date` match hota hai. Un sabhi invoices ka database cloning copy process run hoke new draft invoices initialize ho jati hain aur parent template invoice updates key date updates direct advance state me transform ho jati hain.
- **Overdue Invoice Checker (Daily 9 AM Cron: `0 9 * * *`):** 9 AM automated database query run hoti hai. Jo documents me invoice state status `'sent'` mark ho aur current date due date cross kar gayi ho (`dueDate < Date.now()`), unhe batch command run karke `'overdue'` array settings values save ho jati hain.

📄 Invoice PDF Engine (Server-Side Puppeteer)

Invoices download functionality direct server side backend automation flow script headless browser ke integration runtime PDF create karti hai:

```
// Inside backend/controllers/invoiceController.js -> getInvoicePDF
const browser = await puppeteer.launch({ headless: 'new' });
const page = await browser.newPage();
const html = renderInvoiceHTML(invoiceData); // Custom HTML invoice layout
await page.setContent(html);
const pdfBuffer = await page.pdf({ format: 'A4', printBackground: true });
await browser.close();

// Sending binary stream directly back:
res.setHeader('Content-Type', 'application/pdf');
res.send(pdfBuffer);
```

Headless browser memory me HTML markup content process karke vector layouts standard dynamic A4 size printable PDF stream data generate kar deta hai.

💳 Payment Flow Integration (Razorpay Payment & Verification)

Indian payment system online collections ke liye Razorpay SDK config integration features detailed steps me flow karta hai:

- **Order Generation:** UI order process hit hotey hi controller instance razorpay helper call karke temporary payload key transaction order return karwata hai. `razorpay.orders.create({ amount, currency: "INR" })`
- **Frontend Checkout:** Returns Order ID ko UI payload data verify check ke sath window context Razorpay checkout library frame trigger karke launch loading card open karwa deta hai.
- **Cryptographic Signature check:** User transactions details authenticate karke successful transition callback parameters object status backend signature check verify router hit karwata hai. Signature hash security HMAC algorithm (`crypto.createHmac('sha256', SECRET)`) comparison match verification pass hotey hi database system configuration values `status: 'paid'` transform karke payment event sockets alerts notifications route dispatch.

4. Quick API Cheat Sheet

Method	Endpoint	Auth Check	Controller Module / Purpose
POST	`/api/auth/register`	❌ No Auth	authController.js - Create profile + trigger verification OTP email
POST	`/api/auth/login`	❌ No Auth	authController.js - Validate password + return secure JWT token
GET	`/api/projects`	✅ JWT verified	projectController.js - Fetch projects mapped to request user id
POST	`/api/projects/:id/tasks`	✅ JWT verified	projectController.js - Add task item in subdocument array
GET	`/api/invoices/:id/pdf`	✅ JWT (Admin)	invoiceController.js - Compile & download Puppeteer print invoice
POST	`/api/invoices/:id/razorpay-order`	✅ JWT (Admin)	invoiceController.js - Initializing payment orders gateway object
POST	`/api/ai/proposal`	✅ JWT (Admin)	aiController.js - Google Generative AI proposal drafting